

GUIA PARA PROFESSORES

# Coffee Shop

Passo a passo: HTML, CSS responsivo  
e JavaScript do zero

HTML5

CSS3 · Flexbox · Grid

JavaScript

# Sumário

---

- 1. 1.1 — Esqueleto da página
- 2. 1.2 — Header (título e subtítulo)
- 3. 1.3 — Main e seção “História”
- 4. 1.4 — Seção “Sabores”
- 5. 1.5 — Seção “Fotos” (versão estática)
- 6. 1.6 — Footer (contato)
- 7. 2.1 — Reset, variáveis e fontes
- 8. 2.2 — Body (grid da página)
- 9. 2.3 — Header
- 10. 2.4 — Main e “História”
- 11. 2.5 — Sabores
- 12. 2.6 — Fotos (galeria estática)
- 13. 2.7 — Footer
- 14. 3.1 — Tablet (a partir de 600px)
- 15. 3.2 — Desktop (a partir de 1024px)
- 16. 3.3 — Monitores grandes (a partir de 1600px)
- 17. 4.0 — Preparando o arquivo
- 18. Funcionalidade 1 — Galeria de fotos com preview
- 19. Funcionalidade 2 — Selo “Aberto agora” / “Fechado”
- 20. Funcionalidade 3 — Cardápio gerado por array + laço
- 21. Funcionalidade 4 — Botão “Voltar ao topo”

# Introdução

---

Este guia mostra o passo a passo completo para recriar o site **Coffee Shop** do zero: primeiro a estrutura em HTML, depois a estilização e a responsividade em CSS (mobile-first, com media queries), e por fim as quatro funcionalidades em JavaScript que adicionamos por cima da versão estática.

A ideia é seguir a ordem exata em que a página foi construída. Cada etapa mostra só o trecho de código que deve ser adicionado — os comentários dentro do próprio código explicam o “porquê” de cada decisão.

## Estrutura de pastas

Antes de começar, crie esta estrutura de arquivos e pastas:

```
projeto-coffee/  
├── index.html  
├── icon.ico  
├── imagens/  
│   ├── cafe-graos.jpg  
│   ├── xicaras.jpg  
│   ├── xicaras-vertical.jpg  
│   ├── xicaras-horizontal.jpg  
│   ├── xicaras-horizontal-wide.jpg  
│   ├── barista.jpg  
│   └── foto-a.jpg ... foto-f.jpg  
├── estilos/  
│   ├── style.css  
│   └── media-query.css  
└── js/  
    └── script.js
```

As imagens em `imagens/` podem ser fotos livres de direitos (Unsplash, Pexels) relacionadas a café — o conteúdo exato não importa, só o papel que cada uma cumpre no layout. Os 3 arquivos `xicaras-*.jpg` são recortes diferentes da **mesma foto** ( `xicaras.jpg` , 3 xícaras vistas de cima), feitos num editor de imagens: `xicaras-vertical.jpg` é a foto girada 90° e recortada para caber numa faixa estreita (celular); `xicaras-horizontal.jpg` é um recorte mais justo da foto original (tablet/desktop); `xicaras-horizontal-wide.jpg` é um recorte bem mais largo, com mais mesa visível dos lados (monitores grandes).

---

# Parte 1 — Estrutura HTML

---

Vamos montar o `index.html` por partes. Toda a página fica dividida em 4 blocos principais: `<header>`, `<main>` (com as seções de conteúdo), `<footer>` e, mais tarde, o `<script>`.

---

## 1.1 — Esqueleto da página

Comece com o HTML5 básico: charset, viewport (essencial para responsividade), título e os links para as duas folhas de estilo que vamos criar na Parte 2.

`index.html`

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Coffee Shop</title>
  <link rel="stylesheet" href="estilos/style.css">
  <link rel="stylesheet" href="estilos/media-query.css">
  <link rel="shortcut icon" href="icon.ico" type="image/x-icon">
</head>
<body class="container">

</body>
</html>
```

`viewport` com `width=device-width` é o que permite que as media queries do Passo 3 funcionem corretamente — sem essa linha, celulares tentam renderizar a página como se fosse desktop e depois encolhem tudo.

---

## 1.2 — Header (título e subtítulo)

Dentro do `<body>`, adicione o cabeçalho. A estrutura tem propositalmente duas divs (`.container` e `.item`) por dentro — elas vão servir de “ganchos” para o flexbox de centralização no CSS.

```
<header class="item">

  <div class="container">

    <div class="item">
      <h1>Coffee Shop</h1>
      <p>O melhor café desde 2020</p>
    </div>

  </div>

</header>
```

---

## 1.3 — Main e seção “História”

O `<main>` vai conter todas as seções de conteúdo da página, empilhadas verticalmente. A primeira é a seção “Nossa História”, envolvida por uma `<div>` extra (usada depois para o layout em grid).

```
<main class="item">
  <div>
    <section id="historia">
      <h2>NOSSA HISTÓRIA</h2>
      <p>Lorem ipsum dolor sit, amet consectetur adipisicing elit. Ullam ad vero ipsa at
        quibusdam quisquam ut molestiae incidunt deleniti dolorem, voluptatibus, quo eum est fuga
        consequatur veniam, earum quos aliquam.</p>
      <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Quos aperiam delectus,
        provident doloribus, eos quaerat fuga ratione architecto consequuntur cum nam eaque optio quod ipsum
        nihil ducimus, saepe possimus vitae.</p>
    </section>
  </div>
</main>
```

---

## 1.4 — Seção “Sabores”

Uma seção simples com uma frase de efeito e uma div vazia ( `.foto` ) que vai receber a imagem de fundo (xícaras de café) só via CSS.

```
<section id="sabores">
  <div class="item">
    <p>AROMA MARCANTE E SABOR <br>INESQUECÍVEL</p>
  </div>
  <div class="foto"></div>
</section>
```

---

## 1.5 — Seção “Fotos” (versão estática)

Nesta primeira versão (sem JavaScript), a galeria é só 3 imagens fixas lado a lado.

```
<section id="fotos" class="container">
  
  
  
</section>
```

Repare no `id="f1"` : no CSS, essa primeira foto vai ficar escondida no celular (para a seção não ficar muito comprida) e reaparecer a partir do tablet.

---

## 1.6 — Footer (contato)

Por fim, o rodapé com os dados de contato e uma foto do barista (que só vai aparecer no desktop).

```
<footer class="item">
  <div id="texto">
    <h2>CONTATO</h2>
    <br>
    <p><strong>Telefone:</strong> <br>55 51 4002-8922</p>
    <p><strong>Email:</strong> <br>coffeshop@mail.com</p>
    <p><strong>Endereço:</strong> <br>Av. Bento Gonçalves, 9500 Porto Alegre</p>
  </div>
  
</footer>
```

Com isso, o HTML está completo. Ele ainda vai parecer uma página sem estilo nenhum — é exatamente isso que a Parte 2 resolve.

---

## Parte 2 — CSS base (mobile-first)

Vamos estilizar a página pensando primeiro no celular (telas pequenas). Esse é o conceito de **mobile-first**: escrever o CSS “padrão” para a tela mais estreita, e só depois, na Parte 3, ir *acrescentando* ajustes para telas maiores com media queries.

Todo o CSS desta parte vai para o arquivo `estilos/style.css`.

### 2.1 — Reset, variáveis e fontes

Começamos zerando margens/paddings padrão do navegador, e definindo variáveis CSS (`:root`) para as cores e fontes usadas no site inteiro. Usar variáveis evita repetir códigos de cor no arquivo inteiro e facilita trocar a paleta depois.

`estilos/style.css`

```
@charset "UTF-8";

@import url('https://fonts.googleapis.com/css2?family=Limelight&display=swap');
@import url('https://fonts.googleapis.com/css2?family=Cabin:ital,wght@0,400..700;1,400..700&display=swap');

* {
  margin: 0px;
  padding: 0px;
  box-sizing: border-box;
}

:root {
  --marrom-escuro: #1B1615;
  --marrom-medio: #322F20;
  --marrom-claro: #6A5837;
  --laranja: #F7B538;
  --creme: #FBF2C0;

  --font-titulo: "Limelight", sans-serif;
  --font-texto: "Cabin", sans-serif;
}
```

`box-sizing: border-box` faz com que `padding` e `border` passem a contar *dentro* da largura/altura de um elemento, em vez de somar a ela — evita muita dor de cabeça com elementos “estourando” o layout.

### 2.2 — Body (grid da página)

O `<body>` vira um **grid** de uma coluna só. Como ele não tem altura fixa, cada seção (header, main, footer) ocupa exatamente a altura do seu próprio conteúdo.

```
body {
  display: grid;
}
```

---

## 2.3 — Header

A imagem de fundo dos grãos de café cobre o header inteiro ( `background-size: cover` ) e fica fixa durante a rolagem ( `background-attachment: fixed` ). Um degradê escurece a parte de baixo para o texto ficar legível.

```
header {
  background-image: url(../imagens/cafe-graos.jpg);
  background-attachment: fixed;
  background-size: cover;
  background-position: center center;
  min-height: 60vh;
  color: var(--creme);
}

header .container {
  background-image: linear-gradient(to bottom, rgba(0, 0, 0, 0), var(--marrom-escuro));
  height: 100%;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
}

header .item {
  text-align: center;
  width: 100%;
  max-width: 900px;
  padding: 0 20px;
}

header h1 {
  font-family: var(--font-titulo);
  font-size: 2.4em;
}

header p {
  font-family: var(--font-texto);
  font-size: 1.3em;
}
```

**FLEXBOX:** `display: flex` + `flex-direction: column` + `align-items: center` + `justify-content: center` em `header .container` centralizam o título tanto na horizontal quanto na vertical, em qualquer tamanho de tela.

---

## 2.4 — Main e “História”

O `<main>` também é um grid — dessa vez com 3 linhas (uma para cada seção), cada uma do tamanho do próprio conteúdo ( `auto` ).



```
main {
  display: grid;
  grid-template-rows: auto auto auto;
  grid-template-columns: 100%;
  background-color: var(--marrom-escuro);
}

main > div {
  background-color: var(--marrom-escuro);
  color: var(--creme);
  display: flex;
  justify-content: center;
  padding: 30px 15px;
}

main #historia {
  width: 100%;
  max-width: 300px;
  align-self: center;
}

#historia > h2 {
  font-family: var(--font-titulo);
  font-size: 1.9em;
  text-align: center;
  margin: 20px;
}

#historia > p {
  font-family: var(--font-texto);
  font-size: 1.1em;
  text-align: justify;
}
```

---

## 2.5 — Sabores

No celular a seção é baixa e estreita, e a foto das 3 xícaras (na horizontal) ficava toda cortada nesse formato. Por isso, o texto e a foto são dois blocos empilhados (FLEXBOX em coluna) em vez de um só: primeiro o texto, num fundo liso, depois a foto — usando uma versão **vertical** da mesma imagem ( `xicaras-vertical.jpg` , obtida girando e recortando o arquivo original em um editor de imagens), que se encaixa muito melhor numa faixa estreita.

```
#sabores {
  background-color: var(--marrom-escuro);
  display: flex;
  flex-direction: column;
}

#sabores .item {
  /* Padding-top bem maior: cria uma respiração clara entre o
     último parágrafo de "história" e este texto. */
  padding: 110px 15px 24px;
}

#sabores .foto {
  width: 100%;
  /* aspect-ratio: mesma proporção da foto já cortada, então o
     cover não precisa recortar nada extra – as 3 xícaras aparecem
     inteiras. */
  aspect-ratio: 1920 / 2132;
  /* Degradê: escurece o topo da foto para o texto acima encostar
     sem uma quebra brusca de cor. O segundo stop repetido
     (95%/100% com a mesma cor) evita uma linha de arredondamento
     bem na borda entre os dois blocos. */
  background-image: linear-gradient(to top, rgba(0, 0, 0, 0), var(--marrom-escuro) 95%, var(--
marrom-escuro) 100%), url(../imagens/xicaras-vertical.jpg);
  background-size: cover;
  background-position: center center;
}

#sabores p {
  font-family: var(--font-titulo);
  font-size: 1.6em;
  text-align: center;
  color: var(--creme);
}
```

## 2.6 — Fotos (galeria estática)

No celular, as 3 fotos ficam empilhadas em coluna (flexbox), com a primeira escondida para a seção não ficar longa demais.

```
#fotos {
  background-color: var(--creme);
  display: flex;
  flex-flow: column nowrap;
  align-items: center;
  padding: 20px 0px;
}

#fotos img {
  margin: 10px 0px;
  width: 95%;
  height: 220px;
  object-fit: cover;
  border-radius: 8px;
}

#fotos #f1 {
  display: none;
}
```

---

## 2.7 — Footer

O rodapé usa flexbox em coluna para empilhar o título e os blocos de contato, e esconde a foto do barista (ela só aparece a partir do desktop, na Parte 3).

```
footer {
  background-color: var(--marrom-escuro);
  color: var(--creme);
  min-height: 300px;
  display: flex;
  flex-direction: column;
  justify-content: center;
  padding: 30px 15px;
}

footer > div#texto {
  text-align: center;
  display: flex;
  gap: 15px;
  flex-direction: column;
  justify-content: space-around;
}

footer h2 {
  font-family: var(--font-titulo);
  font-size: 1.8em;
}

footer p {
  font-family: var(--font-texto);
  font-size: 1em;
}

footer .barista {
  display: none;
}
```

Neste ponto, abrindo o `index.html` num celular (ou no modo responsivo do navegador), a página já deve estar com cores, fontes e um layout coerente — só falta ela se adaptar a telas maiores. É isso que a Parte 3 resolve.

---

## Parte 3 — Responsividade (media queries)

---

Criamos um **segundo arquivo**, `estilos/media-query.css` (já referenciado no `<head>` desde a Parte 1), para manter todo o CSS responsivo separado do CSS base — fica mais fácil de navegar do que um arquivo único gigante.

Usamos 3 breakpoints, sempre com `min-width` (ou seja: “a partir desta largura...”), seguindo a lógica mobile-first:

```
/tablet    -> a partir de 600px  
desktop    -> a partir de 1024px  
monitor grande -> a partir de 1600px
```

---

### 3.1 — Tablet (a partir de 600px)

Aumentamos fontes, trocamos a foto vertical de “sabores” pela versão horizontal (já que agora há espaço de sobra), e a galeria de fotos passa de coluna (flexbox) para uma grade de 3 colunas (CSS Grid).

```
estilos/media-query.css
```

```
@media (min-width: 600px) {

  header h1 {
    font-size: 3.4em;
  }

  header p {
    font-size: 1.8em;
  }

  #sabores {
    /* A partir daqui há espaço de sobra para a foto original (na
       horizontal) caber sem cortar as 3 xícaras – maior e mais
       larga, já que a tela também é mais larga. */
    padding: 0;
  }

  #sabores .item {
    padding: 110px 30px 40px;
  }

  #sabores .foto {
    display: block;
    width: 100%;
    aspect-ratio: 2073 / 1037;
    background-image: linear-gradient(to top, rgba(0, 0, 0, 0), var(--marrom-escuro) 95%, var(--
marrom-escuro) 100%), url(../imagens/xicaras-horizontal.jpg);
    background-size: cover;
    background-position: center center;
  }

  #sabores p {
    font-size: 2em;
  }

  #fotos {
    display: grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 20px;
    padding: 30px 20px;
    align-items: center;
  }

  #fotos img {
    margin: 0px;
    width: 100%;
    height: 260px;
  }

  #fotos #f1 {
    display: block;
  }

  footer h2 {
    font-size: 2.2em;
  }

  footer p {
    font-size: 1.3em;
  }
}
```

**GRID:** `grid-template-columns: repeat(3, 1fr)` cria 3 colunas de larguras iguais — é o que coloca as 3 fotos lado a lado a partir do tablet.

---

## 3.2 — Desktop (a partir de 1024px)

Em telas grandes, o header ocupa a tela inteira ( `100vh` ), e o footer vira duas colunas: texto de um lado, foto do barista do outro.

```
@media (min-width: 1024px) {

    header {
        height: 100vh;
        min-height: unset;
    }

    header h1 {
        font-size: 5em;
    }

    header p {
        font-size: 2.4em;
    }

    main #historia {
        max-width: 350px;
    }

    #historia > h2 {
        font-size: 2.2em;
    }

    #historia > p {
        font-size: 1.2em;
    }

    #sabores p {
        font-size: 2.4em;
    }

    #fotos img {
        height: 300px;
    }

    footer {
        height: 600px;
        min-height: unset;
        flex-direction: row;
        align-items: stretch;
        padding: 0;
    }

    footer > div#texto {
        height: auto;
        flex: 1 1 50%;
        padding: 40px;
        text-align: left;
        justify-content: center;
    }

    footer .barista {
        display: block;
        flex: 1 1 50%;
        width: 50%;
        height: 100%;
        object-fit: cover;
        object-position: top center;
    }

    footer h2 {
        font-size: 3em;
    }
}
```

```
    footer p {
      font-size: 1.6em;
    }
  }
}
```

100vh (“100% da altura da tela”) em vez de um valor fixo em pixels garante que o header sempre ocupe a tela inteira do usuário, não importa a resolução do monitor.

### 3.3 — Monitores grandes (a partir de 1600px)

Em monitores muito largos, as fontes do header cresciam pouco em relação ao tamanho da tela. Este breakpoint ajusta isso, faz a seção “História” ocupar a tela inteira, e troca a foto de “sabores” por uma versão ainda mais larga (`xicaras-horizontal-wide.jpg`) — sem isso, a foto ficava com um “zoom” exagerado nas xícaras nesse tipo de tela.

```
@media (min-width: 1600px) {

  header h1 {
    font-size: 6.5em;
  }

  header p {
    font-size: 3em;
  }

  main > div {
    min-height: 100vh;
  }

  main #historia {
    max-width: 450px;
  }

  #sabores .foto {
    aspect-ratio: 2880 / 1037;
    background-image: linear-gradient(to top, rgba(0, 0, 0, 0), var(--marrom-escuro) 95%, var(--marrom-escuro) 100%), url(../imagens/xicaras-horizontal-wide.jpg);
  }

  #historia > h2 {
    font-size: 3em;
  }

  #historia > p {
    font-size: 1.5em;
  }
}
```

**Chegamos à versão final do site sem JavaScript** — um site totalmente estático, responsivo do celular ao monitor ultrawide, usando só HTML e CSS (flexbox e grid). A partir daqui, a Parte 4 adiciona interatividade com JavaScript puro.



## Parte 4 — JavaScript

---

A partir daqui vamos adicionar 4 funcionalidades com JavaScript, na mesma ordem em que foram criadas. Cada uma pratica um conjunto de conceitos: variáveis, arrays, funções, laços de repetição, condicionais e manipulação do DOM.

---

### 4.0 — Preparando o arquivo

Crie o arquivo `js/script.js` (vazio por enquanto) e adicione a tag `<script>` no final do `<body>`, depois de todo o resto do HTML — assim, quando o script rodar, todos os elementos da página já existem.

index.html

```
<script src="js/script.js"></script>
</body>
</html>
```

---

### Funcionalidade 1 — Galeria de fotos com preview

A galeria estática de 3 fotos vira uma galeria interativa: uma foto principal no centro, com botões “<” e “>” para navegar, e prévias das fotos anterior/seguinte levemente apagadas nas laterais.

No celular, essa galeria interativa causava rolagem lateral (fotos na horizontal forçavam a seção a crescer além da largura da tela). A solução foi manter **2 versões** dentro da mesma seção: uma estática (só HTML/CSS, sem JS) para o celular, e a galeria interativa a partir do tablet — cada uma escondida via CSS quando não é a vez dela.

#### HTML — substitua o conteúdo de #fotos

index.html

```

<section id="fotos" class="container">
  <!-- No celular a galeria com JS causava rolagem lateral,
        então aqui mostramos só 2 fotos estáticas (sem botões,
        sem JS). -->
  <div class="fotos-estatico">
    
    
  </div>

  <!-- A partir do tablet a versão estática acima fica
        escondida e esta galeria interativa aparece. -->
  <div class="galeria">
    <button id="btn-anterior" type="button" aria-label="Foto anterior"><</button>
    <div class="fotos-wrap">
      
      
      
    </div>
    <button id="btn-proxima" type="button" aria-label="Próxima foto">></button>
  </div>
</section>

```

## CSS — substitua as regras antigas de #fotos

Remova as regras `#fotos`, `#fotos img` e `#fotos #f1` escritas na Parte 2.6 e coloque estas no lugar, em `estilos/style.css`:

```
estilos/style.css
```

```
#fotos {
  background-color: var(--creme);
  display: flex;
  flex-direction: column;
  align-items: center;
  padding: 30px 15px;
  /* min-width:0 – sem isso, fotos na horizontal (mais largas que
    altas) forçavam esta seção (e a página inteira) a crescer além
    da largura da tela, porque o navegador reserva espaço para a
    largura "natural" da imagem antes do flexbox poder encolhê-la. */
  min-width: 0;
}

#fotos .fotos-estatico {
  display: flex;
  flex-direction: column;
  align-items: center;
  gap: 15px;
  width: 100%;
}

#fotos .fotos-estatico img {
  width: 100%;
  max-width: 500px;
  height: 220px;
  object-fit: cover;
  border-radius: 8px;
}

#fotos .galeria {
  /* Escondida no celular; a media query de 600px troca para "flex". */
  display: none;
  align-items: center;
  justify-content: space-between;
  width: 100%;
  max-width: 650px;
  min-width: 0;
}

#fotos .fotos-wrap {
  /* flex:1 1 auto – sem isso, este bloco só ficava do tamanho dos
    seus filhos, em vez de sempre preencher o espaço entre os dois
    botões (o que fazia a foto principal mudar de largura conforme
    a proporção de cada imagem). */
  flex: 1 1 auto;
  min-width: 0;
  display: flex;
  align-items: center;
  gap: 10px;
}

#fotos img {
  height: 280px;
  object-fit: cover;
  border-radius: 8px;
  display: block;
}

#fotos #foto-galeria {
  flex: 1 1 auto;
  min-width: 0;
}
```

```

#fotos .preview {
  flex: 0 1 65px;
  min-width: 0;
  opacity: 0.75;
}

#foto-anterior {
  /* mask-image cria o efeito degradê: a foto vai ficando
     transparente perto da borda externa (esquerda). */
  -webkit-mask-image: linear-gradient(to left, black 35%, transparent);
  mask-image: linear-gradient(to left, black 35%, transparent);
}

#foto-proxima {
  -webkit-mask-image: linear-gradient(to right, black 35%, transparent);
  mask-image: linear-gradient(to right, black 35%, transparent);
}

#fotos button {
  flex: 0 0 auto;
  width: 32px;
  height: 60px;
  display: flex;
  align-items: center;
  justify-content: center;
  font-size: 1.4em;
  line-height: 1;
  border: none;
  border-radius: 6px;
  background-color: var(--marrom-escuro);
  color: var(--creme);
  cursor: pointer;
}

```

## CSS — ajustes nas media queries

Em `estilos/media-query.css`, dentro do breakpoint de **600px**, troque o antigo bloco `#fotos { display: grid; ... }` por:

```
estilos/media-query.css — dentro de @media (min-width: 600px)
```

```
#fotos .fotos-estatico {
  display: none;
}

#fotos .galeria {
  display: flex;
  max-width: 800px;
}

#fotos img {
  height: 350px;
}

#fotos .preview {
  flex-basis: 90px;
}

#fotos button {
  width: 38px;
  height: 76px;
  font-size: 1.6em;
}
```

E dentro do breakpoint de **1024px**, troque `#fotos img { height: 300px; }` por:

**estilos/media-query.css — dentro de @media (min-width: 1024px)**

```
#fotos .galeria {
  max-width: 1000px;
}

#fotos img {
  height: 460px;
}

#fotos .preview {
  flex-basis: 130px;
}
```

## JavaScript

Agora, o código que faz a galeria funcionar: um array com os caminhos das fotos, uma variável para guardar a posição atual, e duas funções.

**js/script.js**

```
// GALERIA DE FOTOS
// Array com o caminho de cada foto da galeria.
const fotos = [
  "imagens/foto-a.jpg",
  "imagens/foto-b.jpg",
  "imagens/foto-c.jpg",
  "imagens/foto-d.jpg",
  "imagens/foto-e.jpg",
  "imagens/foto-f.jpg"
];

// Variável que guarda o índice (posição no array) da foto atual.
let indiceAtual = 0;

// DOM: pegando os elementos que vamos usar.
const imagemAnterior = document.getElementById("foto-anterior");
const imagemGaleria = document.getElementById("foto-galeria");
const imagemProxima = document.getElementById("foto-proxima");
const botaoAnterior = document.getElementById("btn-anterior");
const botaoProxima = document.getElementById("btn-proxima");

// Função que "corrige" um índice que passou dos limites do array,
// fazendo ele dar a volta (do fim para o começo e vice-versa).
function corrigirIndice(indice) {
  if (indice > fotos.length - 1) {
    return 0;
  }

  if (indice < 0) {
    return fotos.length - 1;
  }

  return indice;
}

// Função que atualiza as 3 imagens (anterior, atual e próxima) de
// acordo com o índice recebido.
function mostrarFoto(indice) {
  const indiceAnterior = corrigirIndice(indice - 1);
  const indiceProximo = corrigirIndice(indice + 1);

  imagemAnterior.src = fotos[indiceAnterior];
  imagemGaleria.src = fotos[indice];
  imagemProxima.src = fotos[indiceProximo];
}

// Ao clicar em "Próxima", avança o índice (com a volta ao início).
botaoProxima.addEventListener("click", function () {
  indiceAtual = corrigirIndice(indiceAtual + 1);
  mostrarFoto(indiceAtual);
});

// Ao clicar em "Anterior", volta o índice (com a volta ao fim).
botaoAnterior.addEventListener("click", function () {
  indiceAtual = corrigirIndice(indiceAtual - 1);
  mostrarFoto(indiceAtual);
});

// Mostra as 3 fotos certas assim que a página carrega.
mostrarFoto(indiceAtual);
```

Conceitos praticados: **array** (lista de fotos), **variável** (índice atual), **função** (reaproveitada duas vezes, para o botão anterior e o próximo), **condicional** (dar a volta no array) e **DOM** (ler e escrever elementos da página).

## Funcionalidade 2 — Selo “Aberto agora” / “Fechado”

Um selo no rodapé que muda de texto e cor automaticamente, de acordo com a hora do relógio do visitante.

### HTML — adicione o selo dentro de #texto, no footer

index.html

```
<h2>CONTATO</h2>
<span id="selo-horario" class="selo"></span>
<br>
```

### CSS

estilos/style.css

```
.selo {
  /* align-self:center – sem isso, o span esticaria e viraria uma
     barra ocupando 100% da largura (comportamento padrão de um
     item dentro de um flex column). */
  align-self: center;
  font-family: var(--font-texto);
  font-size: 0.9em;
  font-weight: bold;
  padding: 5px 14px;
  border-radius: 999px;
}

.selo-aberto {
  background-color: var(--laranja);
  color: var(--marrom-escuro);
}

.selo-fechado {
  background-color: var(--marrom-claro);
  color: var(--creme);
}
```

E, dentro do breakpoint de 1024px em `media-query.css` (onde o footer vira duas colunas com o texto alinhado à esquerda):

estilos/media-query.css — dentro de @media (min-width: 1024px)

```
.selo {
  align-self: flex-start;
}
```

### JavaScript

js/script.js

```
// SELO "ABERTO AGORA" / "FECHADO"
// Variáveis com o horário de funcionamento (formato 24h).
const horaAbertura = 8;
const horaFechamento = 20;

// Tratamento de dados: pega a data/hora atual do visitante e extrai
// só a hora (um número de 0 a 23).
const horaAtual = new Date().getHours();

const seloHorario = document.getElementById("selo-horario");

// Condicional: a loja está aberta se a hora atual estiver entre o
// horário de abertura (incluso) e o de fechamento (não incluso).
if (horaAtual >= horaAbertura && horaAtual < horaFechamento) {
  seloHorario.textContent = "Estamos abertos agora ☺";
  seloHorario.classList.add("selo-aberto");
} else {
  seloHorario.textContent = "Fechado no momento – abrimos às " + horaAbertura + "h";
  seloHorario.classList.add("selo-fechado");
}
```

Conceitos praticados: **tratamento de dados** (o objeto `Date` do JavaScript), **operadores de comparação** (`>=`, `<`) combinados com `&&`, e **condicional** `if/else`.

## Funcionalidade 3 — Cardápio gerado por array + laço

Uma seção nova, cujo conteúdo não é escrito no HTML — um array de objetos é percorrido por um laço `for`, que cria e insere cada item na página.

### HTML — nova seção, entre “sabores” e “fotos”

index.html

```
<section id="cardapio">
  <h2>NOSSO CARDÁPIO</h2>
  <div id="cardapio-lista"></div>
</section>
```

Como agora o `<main>` tem 4 seções em vez de 3, atualize `grid-template-rows` em `style.css` de `auto auto auto` para `auto auto auto auto`.

## CSS

estilos/style.css



```
#cardapio {
  background-color: var(--marrom-medio);
  color: var(--creme);
  padding: 40px 20px;
}

#cardapio h2 {
  font-family: var(--font-titulo);
  font-size: 1.9em;
  text-align: center;
  margin-bottom: 24px;
}

#cardapio-lista {
  display: grid;
  grid-template-columns: 1fr;
  gap: 20px;
  max-width: 1000px;
  margin: 0 auto;
}

.item-cardapio {
  background-color: var(--marrom-escuro);
  border-radius: 8px;
  padding: 20px;
}

.item-cardapio h3 {
  font-family: var(--font-titulo);
  font-size: 1.3em;
  margin-bottom: 6px;
}

.item-cardapio .preco {
  font-family: var(--font-texto);
  font-weight: bold;
  color: var(--laranja);
  margin-bottom: 8px;
}

.item-cardapio .descricao {
  font-family: var(--font-texto);
  font-size: 0.95em;
}
```

E, para o cardápio ganhar mais colunas em telas maiores, em `media-query.css`:

**estilos/media-query.css — dentro de @media (min-width: 600px)**

```
#cardapio-lista {
  grid-template-columns: repeat(2, 1fr);
}
```

**estilos/media-query.css — dentro de @media (min-width: 1024px)**

```
#cardapio-lista {
  grid-template-columns: repeat(3, 1fr);
}
```

## JavaScript

js/script.js

```
// CARDÁPIO DE CAFÉS
// Array de objetos: cada item do cardápio tem nome, preço e descrição.
const cardapio = [
  { nome: "Espresso", preco: 6.5, descricao: "Café puro, forte e encorpado." },
  { nome: "Cappuccino", preco: 9.0, descricao: "Espresso, leite vaporizado e espuma cremosa." },
  { nome: "Latte", preco: 9.5, descricao: "Espresso com bastante leite vaporizado e uma fina camada de espuma." },
  { nome: "Mocha", preco: 10.5, descricao: "Espresso, chocolate quente e leite vaporizado." },
  { nome: "Chá Gelado", preco: 7.0, descricao: "Chá preto gelado, servido com rodela de limão." },
  { nome: "Croissant", preco: 8.0, descricao: "Croissant amanteigado, assado na hora." }
];

const listaCardapio = document.getElementById("cardapio-lista");

// Função que recebe um item do cardápio (um objeto do array acima) e
// devolve o elemento HTML pronto do card correspondente.
function criarItemCardapio(item) {
  const card = document.createElement("div");
  card.className = "item-cardapio";

  const titulo = document.createElement("h3");
  titulo.textContent = item.nome;

  const preco = document.createElement("p");
  preco.className = "preco";
  // Tratamento de dados: formata o número (ex: 9.5) como preço em
  // reais (ex: "R$ 9,50").
  preco.textContent = "R$ " + item.preco.toFixed(2).replace(".", ",");

  const descricao = document.createElement("p");
  descricao.className = "descricao";
  descricao.textContent = item.descricao;

  card.appendChild(titulo);
  card.appendChild(preco);
  card.appendChild(descricao);

  return card;
}

// Repetição: percorre o array do cardápio e adiciona cada card na
// página, um por um.
for (let i = 0; i < cardapio.length; i++) {
  const cardHTML = criarItemCardapio(cardapio[i]);
  listaCardapio.appendChild(cardHTML);
}
```

Conceitos praticados: **array de objetos**, **repetição** ( `for` ), **função com retorno**, **DOM** ( `createElement` / `appendChild` ) e **tratamento de dados** ( `toFixed()` e `replace()` ) para formatar o preço).

---

## Funcionalidade 4 — Botão “Voltar ao topo”

Um botão fixo no canto da tela, que só aparece depois que o usuário rola a página, e que a leva de volta ao topo suavemente ao ser clicado.

### HTML — no final do body, antes do script

index.html

```
<button id="btn-topo" type="button" aria-label="Voltar ao topo">▲</button>

<script src="js/script.js"></script>
```

### CSS

estilos/style.css

```
#btn-topo {
  /* Escondido por padrão; o JS adiciona a classe .visivel depois
     que a página é rolada. */
  display: none;
  position: fixed;
  right: 20px;
  bottom: 20px;
  width: 44px;
  height: 44px;
  border: none;
  border-radius: 50%;
  background-color: var(--marrom-escuro);
  color: var(--creme);
  font-size: 1.3em;
  cursor: pointer;
  z-index: 100;
}

#btn-topo.visivel {
  display: flex;
  align-items: center;
  justify-content: center;
}
```

### JavaScript

js/script.js

```
// BOTÃO "VOLTAR AO TOPO"
const botaoTopo = document.getElementById("btn-topo");

// Evento de rolagem: toda vez que o usuário rola a página, verifica
// a posição atual (window.scrollY) e decide se mostra o botão.
window.addEventListener("scroll", function () {
  if (window.scrollY > 400) {
    botaoTopo.classList.add("visivel");
  } else {
    botaoTopo.classList.remove("visivel");
  }
});

// Ao clicar, rola a página de volta para o topo suavemente.
botaoTopo.addEventListener("click", function () {
  window.scrollTo({ top: 0, behavior: "smooth" });
});
```

Conceitos praticados: **evento** de rolagem ( `scroll` ) além do clique, **condicional** comparando a posição da rolagem, e **DOM** ( `classList` ).

---

# Conclusão

---

Com essas 4 funcionalidades, chegamos à versão final do site: uma página totalmente responsiva (celular, tablet, desktop e monitores grandes), construída com HTML semântico, CSS moderno (variáveis, flexbox, grid, media queries) e JavaScript puro, sem nenhuma biblioteca externa.

Sugestões de exercício para os alunos, depois de acompanharem este guia:

- Trocar as cores do site editando só as variáveis em `:root`.
- Adicionar um novo item ao array `cardapio` e ver o card aparecer sozinho.
- Mudar o horário de abertura/fechamento e testar o selo em horários diferentes.
- Adicionar uma 4ª foto de preview, ou trocar a ordem do array `fotos`.